



Bases de Datos

Clase: Propiedades de SQL

Susana Medina Gordillo
susana.medina@correounivalle.edu.co

Estudiantes

Atributos

Tuplas

The diagram illustrates the structure of a database table. The word 'Atributos' (Attributes) is positioned above the table headers, with four arrows pointing to the columns: 'codEst', 'nombre', 'apellido', and 'edad'. The word 'Tuplas' (Tuples) is positioned to the left of the table rows, with eight arrows pointing to each row, representing individual student records.

codEst	nombre	apellido	edad	ciudad
0011	Juan	Dominguez	17	Cali
0012	Mario	Rodriguez	21	Yumbo
0013	David	Santos	20	Cali
0014	Ximena	Caicedo	18	Cali
0015	Maria	Perez	16	Cali
0016	Felipe	Holguín	23	Yumbo
0017	Juliana	Fernandez	17	Palmira
0018	Andrés	Perez	19	Jamundí

Ejemplo: Base de datos de una empresa

CLIENTES (codcli, nombre, direccion, codpostal, codciu)

VENDEDORES (codven, nombre, direccion, codpostal, codpciu, codjefe)

CIUDADES (codciu, nombre, coddep)

DEPARTAMENTOS (coddep, nombre)

ARTICULOS (codart, descrip, precio, stock, stock_min, dto)

FACTURAS (codfac, fecha, codcli, codven, iva, dto)

LINEAS_FAC (codfac, linea, cant, codart, precio, dto)

Restricciones de Integridad



CLIENTES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codjefe → **VENDEDORES** : Acepta nulos.

CIUDADES coddep → **DEPARTAMENTOS**: Acepta nulos.

FACTURAS codcli → **CLIENTES** : Acepta nulos.

FACTURAS codven → **VENDEDORES** : Acepta nulos.

LINEAS_FAC codfac → **FACTURAS** : No acepta nulos.

LINEAS_FAC codart → **ARTICULOS** : No acepta nulos.

Vistas (VIEW)

—

Vistas (VIEW)



Las relaciones definidas con la sentencia **CREATE TABLE** realmente existen en la base de datos, con una organización física. Las tablas son **persistentes**.

Hay otra clase de relaciones de SQL, llamadas **vistas**, que no existen físicamente y se definen con una expresión muy **similar** a una consulta. Las vistas se pueden consultar como si existieran físicamente, y en algunos casos hasta es posible modificarlas.



Definición de Vistas

```
CREATE VIEW nombre_vista
AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Vistas (VIEW) II



Las vistas poseen **filas** y **columnas** similar a las tablas reales.

Para definir una vista pueden usarse funciones SQL, sentencias como **WHERE** y **JOIN** de modo que parezca que los datos provienen de una sola tabla.

Vistas: Ejemplo 1

Se desean tener acceso constante a los clientes de **Cali**.

```
CREATE VIEW vista_Cli_Cali AS  
SELECT *  
FROM Clientes  
WHERE codciu=2  
;
```

codciu=2 es
Cali

Vistas: Ejemplo 2



Se desean tener acceso constante a las facturas de los clientes de **Cali**.

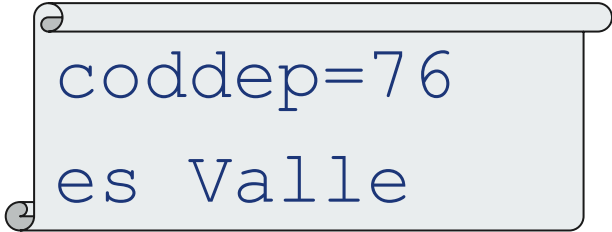
```
CREATE VIEW vista_VF_Cali AS  
SELECT *  
FROM Clientes , Facturas, Ciudades  
WHERE  
Clientes.codpciu=Ciudades.codciu  
AND Clientes.codcli=Facturas.codcli  
AND codciu="2" ;
```

Vistas: Ejemplo 3



Se desean tener acceso constante a los vendedores del **Valle del Cauca**.

```
CREATE VIEW Ven_Valle (nom, dir) AS
SELECT Vendedores.nombre, Vendedores.direccion
FROM Vendedores, Ciudades
WHERE Vendedores.codpciu=Ciudades.codciu
      AND coddep="76"
;
```



```
coddep=76
es Valle
```

Vistas: Ejemplo 3 INNER JOIN



Se desean tener acceso constante a los vendedores del **Valle del Cauca**.

```
CREATE VIEW Ven_Valle (nom, dir) AS
SELECT Vendedores.nombre, Vendedores.direccion
FROM Vendedores INNER JOIN Ciudades
ON Vendedores.codpciu=Ciudades.codciu
    AND coddep="76"
;
```

Vistas: Ejemplo 3 INNER JOIN + subconsulta



Se desean tener acceso constante a los vendedores del **Valle del Cauca**.

```
CREATE VIEW Ven_Valle (nom, dir) AS
SELECT Vendedores.nombre, Vendedores.direccion
FROM Vendedores INNER JOIN Ciudades
ON Vendedores.codpciu=Ciudades.codciu
    AND coddep IN (SELECT coddep FROM
Departamentos WHERE nombre="Valle")
;
```

Modificación de vistas



En algunas circunstancias es posible ejecutar una **inserción**, **eliminación** o **actualización** en una **vista**. Esto parece absurdo ya que la vista no existe como la tabla base.

En muchas vistas esto no puede hacerse. Pero en vistas simples, llamadas **vistas actualizables**, se puede traducir la modificación de la vista a una modificación equivalente de la **tabla base**.

Se admiten modificaciones sobre vistas que se definen seleccionando (por medio de **SELECT**, no de **SELECT DISTINCT**) **algunos atributos** provenientes de la relación R (que también puede ser una vista actualizable).

Actualizando una Vista



Sobre una vista ya existente podemos actualizarla:

```
CREATE OR REPLACE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Ejemplo:

```
CREATE OR REPLACE VIEW vista_Cli_Cali AS  
SELECT nombre, direccion  
FROM Clientes  
WHERE codciu=2;
```



¿Cómo se borra una Vista?

—

Vistas: Eliminar (DROP)



```
DROP VIEW view_name;
```

Ejemplos:

```
DROP VIEW vista_VF_Cali ;
```

```
DROP VIEW vista_Cli_Cali ;
```

```
DROP VIEW Ven_Valle ;
```

Propiedades SQL

Restricciones de dominio

Restricciones de tabla base

Restricciones generales (aserciones)



Restricciones en SQL



La **integridad** de los datos en una base de datos es un aspecto muy importante a tener en cuenta en la implementación de un diseño. Existen al menos dos formas de mantenerla:

- Incorporando las **restricciones** en los programas de aplicación (se multiplica el código específico que implementa restricciones en las aplicaciones).
- Incorporando **declaraciones** al SMBD, formando parte del esquema de la base de datos.

Restricciones de Dominio



Es una restricción en SQL que se aplica a toda la columna definida en el dominio en cuestión.

Ejemplo:

```
CREATE DOMAIN Color CHAR(6) DEFAULT "???"  
CONSTRAINT Colores_Validos  
CHECK ( VALUE IN  
        ("Rojo", "Azul", "Amarillo",  
        "Verde", "???" )  
);
```

Cláusula CHECK



La cláusula **check** de SQL puede aplicarse a **declaraciones** de relaciones y a declaraciones de dominios. Cuando se aplica a declaraciones de relaciones, la cláusula **check(P)** especifica un predicado **P** que deben cumplir **todas las tuplas de la relación**.

Un uso frecuente de la cláusula check es garantizar que los valores de los atributos cumplan las **condiciones** especificadas, lo que permite en realidad construir un potente sistema de tipos.

Restricciones de Tablas Base



Cualquiera de las siguientes es **restricción** de tabla base en SQL:

- Definición de **llave** candidata
- Definición de **llave** externa
- Definición de restricción de **verificación**

Estas restricciones se definen por medio de la sentencia **CONSTRAINT** <nombre de la restricción>

Restricciones de Tablas Base

Llaves candidata

Puede tener alguna de las siguientes formas:

UNIQUE <lista de los nombres de los atributos/columnas separados por comas>

PRIMARY KEY <lista de los nombres de los atributos/columnas separados por comas>

La lista **NO** puede estar vacía!

Restricciones de Tablas Base

Llaves externas/foráneas

FOREIGN KEY <lista de los nombres de los atributos/columnas separados por comas> **REFERENCES** <nombre de la tabla base>

[<lista de los nombres de los atributos/columnas separados por comas>]

[ON DELETE <acción referencial>]

[ON UPDATE <acción referencial>]

Restricciones de Tablas Base

Llaves externas/foráneas

FOREIGN KEY <lista de los nombres de los atributos/columnas separados por comas>

REFERENCES <nombre de la tabla base>

[<lista de los nombres de los atributos/columnas separados por comas>]

[**ON DELETE** <acción referencial>]

[**ON UPDATE** <acción referencial>]

NO ACTION
CASCADE
SET DEFAULT
SET NULL

Acciones Referenciales



Política en cascada (CASCADE)

Cuando se borran o actualizan tuplas en la tabla que posee la llave primaria, se **borran** o se **actualizan** las tuplas relacionadas en la tabla que posee la llave foránea.

Política de asignar valor nulo (SET NULL)

En la cual ante retiros o modificaciones de tuplas en la tabla que posee la llave primaria, se modifican los valores de la tabla que maneja la llave foránea, cambiándolos a **NULL**.

Política de asignar valor por defecto (SET DEFAULT)

En la tabla que posee la llave primaria, se modifican los valores de la tabla que maneja la llave foránea cambiándolos al valor **DEFAULT**.

Restricciones de Tablas Base

Restricciones de verificación



CHECK (<expresión condicional>)

La condición es una expresión lógica o predicado booleano y puede ser compleja o no.

Ejemplo: no queremos permitir ingresen valores nulos.

CHECK <nombre_columna> **IS NOT NULL**) al momento de crear la tabla (**CREATE TABLE**).

Restricciones Generales I



Las restricciones generales o aserciones (ASSERTION) son predicados que expresa una **condición** que la base de datos debe satisfacer siempre. Las restricciones de dominio y las de integridad referencial son formas especiales de las aserciones.

Estos tipos de aserciones permiten verificar con **facilidad** y pueden afectar a una gran variedad de aplicaciones de bases de datos.

Restricciones Generales II



Hay muchas restricciones que no se pueden expresar utilizando únicamente estas formas especiales.

Ejemplos de estas restricciones son:

- La **suma** de los importes de todos los **préstamos** de cada **sucursal** debe ser **menor** que la suma de los **saldos** de todas las cuentas de esa sucursal.
- Cada préstamo tiene al menos un cliente que tiene una cuenta con un **saldo mínimo** de \$1.000.000

Restricciones Generales III



Cuando se crea una aserción, el sistema comprueba su validez. Si la aserción es **válida**, sólo se permiten las **modificaciones posteriores** de la base de datos que no hagan que se viole la aserción.

Esta comprobación puede introducir una **sobrecarga** importante si se han realizado aserciones complejas. Por tanto, las aserciones deben utilizarse con mucha **cautela**

ASSERTION: Sintaxis



En SQL las aserciones adoptan la forma:

```
CREATE ASSERTION <nombreA>  
CHECK (<predicado>)
```

Y para eliminarlo:

```
DROP ASSERTION <nombreA>
```

ASSERTION: Ejemplo 1



```
CREATE ASSERTION restriccion_suma
CHECK (NOT EXISTS
  (SELECT * FROM Sucursal WHERE (SELECT
    SUM(importe) FROM Prestamo
    WHERE Prestamo.nombre_sucursal =
    Sucursal.nombre_sucursal)
    >= (SELECT SUM(saldo) FROM Cuenta
    WHERE Cuenta.nombre_sucursal =
    Sucursal.nombre_sucursal)))
```


ASSERTION: Ejemplo 2



```
CREATE ASSERTION restriccion_saldo CHECK
(NOT EXISTS (SELECT * FROM Prestamo
             WHERE NOT EXISTS SELECT *
FROM Prestatario, Impositor, Cuenta
WHERE Prestamo.numero_prestamo =
Prestatario.numero_prestamo
AND Prestatario.nombre_cliente =
Impositor.nombre_cliente
AND Impositor.numero_cuenta =
Cuenta.numero_cuenta
AND Cuenta.saldo >= 1000000)))
```

Receso 15 min





Práctica